University of Messina
Department of Mathematical and Computer Sciences
Physical and Geosciences

The Tesla Roadster "SpaceX Package" Numerical Simulation of the Cold-Gas "Flight"

Course: Data Modelling
Professor: Sebastiano Vasi
Student: Dana Abdrakhmanova

The Illusion of Flight: Breaking Down the Viral Stunt

When Elon Musk unveiled the highly anticipated next-generation Tesla Roadster, the presentation culminated in a moment that seemed to defy gravity. The vehicle lifted off the stage floor, hovered momentarily, performed a mid-air flip, and landed softly, resembling a scene straight out of a science fiction film. For the general public, it was a spectacle of futuristic engineering. However, for physicists and engineers, it raised a fundamental question: *How does a two-tonne vehicle achieve sustained flight without wings?*

The answer reveals that the Roadster does not "fly" in the traditional aeronautical sense. It lacks fixed wings to generate aerodynamic lift, and it lacks propellers or jet turbines for sustained propulsion. Instead, the vehicle operates as a highly controlled, multi-directional projectile. It achieves its airborne state by literally pushing off the ground using the principles of reaction thrust, combined with an extreme adherence to the ground prior to takeoff. This engineering feat is not a step toward mass-market flying cars, but rather a fascinating study of extreme physics applied to automotive chassis dynamics.



Link for official 45 seconds video-presentation

The Three Pillars of the SpaceX Package

To understand the physics behind this stunt, we must deconstruct the vehicle's "SpaceX Package" into three distinct, highly advanced subsystems that work in tandem:

1. **The Aerodynamic "Vacuum Cleaner" (Downforce over Lift):** Before a car can push off the ground, it must first be capable of sticking to it with immense force. Tesla implemented a multi-mode suction system—essentially four high-powered fans (20 kW each) built into the floor. By extracting air from beneath the car through an aerodynamic skirt, the system creates a significant negative pressure differential. This results in approximately 4.7 tonnes (about 10,360 lbs) of artificial downforce. Physically, this does not lift the car; it drastically increases the normal force, allowing the tires to maintain traction even when driving upside down (as demonstrated in the "loop-the-loop" segment).
2. **SpaceX Cold-Gas Thrusters (Reaction Thrust):** The actual lifting force is generated by cold-gas propulsion. Hidden within the vehicle's frame are two carbon-fiber pressure vessels holding a total of 600 liters of compressed nitrogen (or air) at an extreme pressure of roughly 690 bar (10,000 psi)—about 250 times the pressure of a standard car tire. When flight mode is engaged, this gas is expelled downwards through ten strategically placed nozzles. According to Newton's Third Law of Motion, the rapid expulsion of gas mass downwards generates an equal and opposite force upwards. This system is capable of producing up to 80 kilonewtons (≈18,000 lbs) of thrust, which is sufficient to overcome the vehicle's weight (approx. 2,000 kg) and lift it into the air for a very brief period.
3. **The Megawatt Powertrain:** Underpinning all these systems is a tri-motor electric powertrain delivering over 1 megawatt (1,341 hp) of power. This provides the immense energy required to run the vacuum fans and control the thrusters, while simultaneously ensuring that the car remains one of the fastest accelerating production vehicles in the world when grounded.

The Core Problem: Marketing Claims vs. Physical Reality

Tesla's official statements suggest that the gas supply allows the Roadster to stay airborne for approximately 6.7 seconds. However, these figures are often calculated under ideal, frictionless laboratory conditions. In reality, the vehicle operates in a fluid medium (the atmosphere) and is subject to multiple resistive forces.

As the car moves vertically, it encounters aerodynamic drag, which increases with the square of its velocity. Furthermore, as the compressed gas is exhausted, the mass of the vehicle decreases (due to the loss of the gas mass), and the pressure inside the tank drops, altering the exhaust velocity over time. Finally, when the gas supply is depleted, the vehicle transitions from a state of powered hover to pure free-fall under the influence of gravity.

$$F_D = \frac{1}{2} C \rho A v^2$$

F_D = drag force
C = drag coefficient (dimensionless)
ρ = fluid density
A = reference area
v = flow velocity relative to the object

Therefore, a critical engineering audit is required to determine the true flight envelope of the vehicle.

Project Objectives and Methodology

The objective of this project is to conduct an independent, computationally driven physical audit of the Tesla Roadster's flight capabilities. Rather than relying on marketing figures, we will build a numerical model to simulate the vehicle's dynamics based on strict physical laws.

Using Python as our computational tool, this project will achieve three main objectives:

- **Thermodynamic Analysis:** We will apply the Ideal Gas Law ($PV=mRT$) and the Saint-Venant–Wantzel equation for adiabatic expansion to determine the mass of the compressed gas, its exhaust velocity, and the maximum theoretical thrust.
- **Kinematic Simulation:** We will formulate the equation of motion (Newton's Second Law) as an Ordinary Differential Equation (ODE). Using scipy.integrate.solve_ivp, we will simulate the altitude and vertical velocity of the car over time, observing what happens at the moment the gas runs out.
- **Optimization & Parameter Space Exploration:** We will utilize scipy.optimize to determine how the vehicle's mass and the tank's volume influence the hover duration. Furthermore, we will generate 3D surface plots using matplotlib to visualize the complex relationship between mass, gas volume, and flight time.

By the end of this report, we will have transformed a cinematic stunt into a rigorous mathematical model, demonstrating the power of Data Modelling and Python (NumPy, SciPy, Matplotlib) in solving complex, multi-disciplinary engineering problems.

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from scipy.optimize import brentq
from scipy import constants
```

Code part 1. Libraries

Simulation Parameters and Physical Constants

The foundation of our numerical model relies on defining the precise physical boundary conditions. To accurately simulate the flight dynamics of the Tesla Roadster, the Python script initializes a set of parameters that represent the vehicle's inertial and aerodynamic properties. The mass of the vehicle is set to 2,000 kg, representing the estimated weight of the regular car. To model the aerodynamic resistance during the ascent and descent, we define the coefficient of drag equals to 0.30, the vehicle's frontal area to 2.0 square meters, and the density of the surrounding air 1.225*kg*/m^3 at 15°C.

To demonstrate the principle and avoid unnecessary computational loads, we simulate a scaled-down 100-liter tank at 300 bar. The actual 600-liter tank at 690 bar would hold roughly 6 times more gas, aligning closer to Tesla's 6.7-second claim.

```python
m_car = 2000.0
Cd = 0.30
A = 2.0
rho_air = 1.225

V_tank = 0.1
P_tank = 300e5
T_tank = 293.15
P_atm = constants.atm

gamma = 1.4
M_gas = 0.02897
R = constants.R
R_spec = R / M_gas
g = constants.g
```

Code part 2. Constants

The second group of parameters defines the cold-gas thruster system, which acts as our propulsion source. In this specific simulation run, we model a 100-liter tank pressurized to 300 bar at room temperature. To calculate the actual thrust, the script defines the thermodynamic properties of the expelled gas (modeled as standard air/nitrogen). We utilize the heat capacity ratio and the molar mass of the gas to compute the specific gas constant. These thermodynamic

variables are essential for determining the mass of the stored gas via the Ideal Gas Law and for calculating the exhaust velocity using the Saint-Venant–Wantzel equation.

Finally, the script imports the standard gravitational acceleration directly from scipy.constants. By structuring the code this way, we maintain a clear separation between the variable parameters (like tank pressure and vehicle mass) and the immutable physical constants. This modular approach allows us to easily modify the system's characteristics—for instance, testing how a larger tank or a lighter vehicle affects the hover time—without rewriting the core mathematical functions.

Feasibility Analysis and Thermodynamic Constraints

This specific section of the computational model serves as the foundational feasibility check for the Tesla Roadster's flight system. Before attempting to simulate the complex trajectory of the vehicle, it is necessary to determine the absolute physical limits imposed by the cold-gas thruster design. By calculating the thermodynamic properties of the compressed gas and the required upward force, this code establishes whether the car can actually leave the ground under the given parameters. It acts as a strict reality check against marketing claims, translating the engineering specifications of the gas tank and nozzles into tangible physics metrics.

The calculation evaluates the system through a logical, step-by-step physical breakdown. First, it determines how fast the gas is expelled from the nozzles, which dictates the overall efficiency of the thrust. Next, it calculates the total mass of gas available in the tank and the exact force required to overcome Earth's gravity for a two-tonne vehicle. Finally, by dividing the total available gas by the required mass flow rate, the model outputs the maximum theoretical hover time. This calculation directly demonstrates the core limitation of the system: the gas supply depletes almost instantly. It mathematically proves that sustained flight is impossible for this vehicle, confirming that the demonstration is merely a brief, controlled ballistic maneuver rather than true aviation.

```
v_exhaust = np.sqrt((2 * gamma / (gamma - 1)) * R_spec * T_tank *
                    (1 - (P_atm / P_tank) ** ((gamma - 1) / gamma)))
print(f"Exhaust velocity: {v_exhaust:.2f} m/s")

m_gas = (P_tank * V_tank) / (R_spec * T_tank)
print(f"Mass of compressed gas in tank: {m_gas:.2f} kg")

F_required = m_car * g
print(f"Required thrust to lift the car: {F_required:.2f} Newtons")

m_dot_required = F_required / v_exhaust
print(f"Required gas mass flow rate: {m_dot_required:.2f} kg/s")

t_hover = m_gas / m_dot_required
print(f"Car hover time: {t_hover:.2f} seconds! (Spoiler: very short)")
```

```
Exhaust velocity: 687.81 m/s
Mass of compressed gas in tank: 35.66 kg
Required thrust to lift the car: 19613.30 Newtons
Required gas mass flow rate: 28.52 kg/s
Car hover time: 1.25 seconds! (Spoiler: very short)
```

Code Part 3. Calculation of thrust and flight time

Dynamic Flight Simulation (ODE)

While the previous calculations determined the theoretical maximum hover time, they did not show exactly how the vehicle behaves during the flight. To capture the continuous motion of the car—from liftoff to the inevitable fall—the model employs an Ordinary Differential Equation (ODE) based on Newton's Second Law of Motion. This simulation tracks the vehicle's altitude and vertical velocity simultaneously, updating both variables at every fraction of a second to generate a complete flight profile.

The core of the simulation is a custom function that calculates the net acceleration of the vehicle by evaluating all acting forces. It accounts for the constant downward pull of gravity, the upward thrust from the gas nozzles, and the aerodynamic drag. The drag force is particularly important because it dynamically opposes the car's movement and increases significantly as the vehicle's speed increases. The function also introduces a crucial conditional logic: thrust is only applied while the gas supply lasts. Once the calculated burn time is reached, the thrust drops instantly to zero, and the vehicle becomes a passive object in free-fall.

To solve this continuous problem, the script utilizes the solve_ivp function from the SciPy library, specifically employing a Runge-Kutta numerical integration method. This solver takes the initial state of the car (resting on the ground) and calculates its trajectory over a defined time span. The resulting data provides a precise, second-by-second visualization of the stunt, clearly demonstrating the brief period of powered ascent followed by the rapid descent, thereby mathematically proving the ephemeral nature of the vehicle's flight capability.

```
def flight_dynamics(t, state, t_burn):
    y, v = state
    if t < t_burn:
        thrust = F_required * 1.2
    else:
        thrust = 0.0
    drag = 0.5 * rho_air * Cd * A * v * np.abs(v)
    a = (thrust - m_car * g - drag) / m_car
    return [v, a]

t_span = (0, max(15, t_hover + 5))
t_eval = np.linspace(t_span[0], t_span[1], 500)
initial_state = [0.0, 0.0]
sol = solve_ivp(flight_dynamics, t_span, initial_state,
                t_eval=t_eval, args=(t_hover,))
```

Code Part 4. Flight Simulation (Differential Equation)

Visualization of the Flight Trajectory (Code Explanation)

After computing the vehicle's motion using the differential equation solver, the next crucial step is to visually interpret the results. This section of the code utilizes the Matplotlib library to translate raw numerical data into a clear, scientific graph. By plotting the altitude against time, we can visually demonstrate the entire flight envelope of the Roadster. The code specifically

plots the altitude trajectory as a solid red line, making it easy to track the vehicle's vertical displacement.

To provide context to the trajectory, the code includes specific visual markers for critical physical events. A vertical dashed black line is drawn at the exact moment the fuel supply is exhausted, visually separating the powered phase of the flight from the free-fall phase. Additionally, a horizontal gray line represents the ground level. Finally, the code applies standard scientific formatting—including axis labels, a descriptive title, and a grid—to ensure the plot is publication-ready and easily readable, fulfilling the requirement for high-quality data presentation.

```
plt.figure(figsize=(10, 5))
plt.plot(sol.t, sol.y[0], color='red', linewidth=2, label='Altitude')
plt.axvline(x=t_hover, color='black', linestyle='--',
            label='The gas has run out (Gravity takes over)')
plt.axhline(y=0, color='gray', linestyle='-')

plt.title('Simulation of a Tesla Roadster flying using gas-powered engines')
plt.xlabel('Time (s)')
plt.ylabel('Height above the ground (m)')
plt.grid(True, linestyle=':', alpha=0.7)
plt.legend()
plt.show()
```

Code Part 5. Flight visualization

Data Interpretation and Kinematic Analysis (Graph Analysis)

The resulting graph provides a stark, undeniable visualization of the physical limitations of the cold-gas thruster system. During the initial powered phase (before the dashed line), the vehicle's altitude follows a steep parabolic trajectory. Because we modeled the thrust to be 120% of the vehicle's weight, the Roadster experiences a net positive force, allowing it to actively climb and gain altitude.

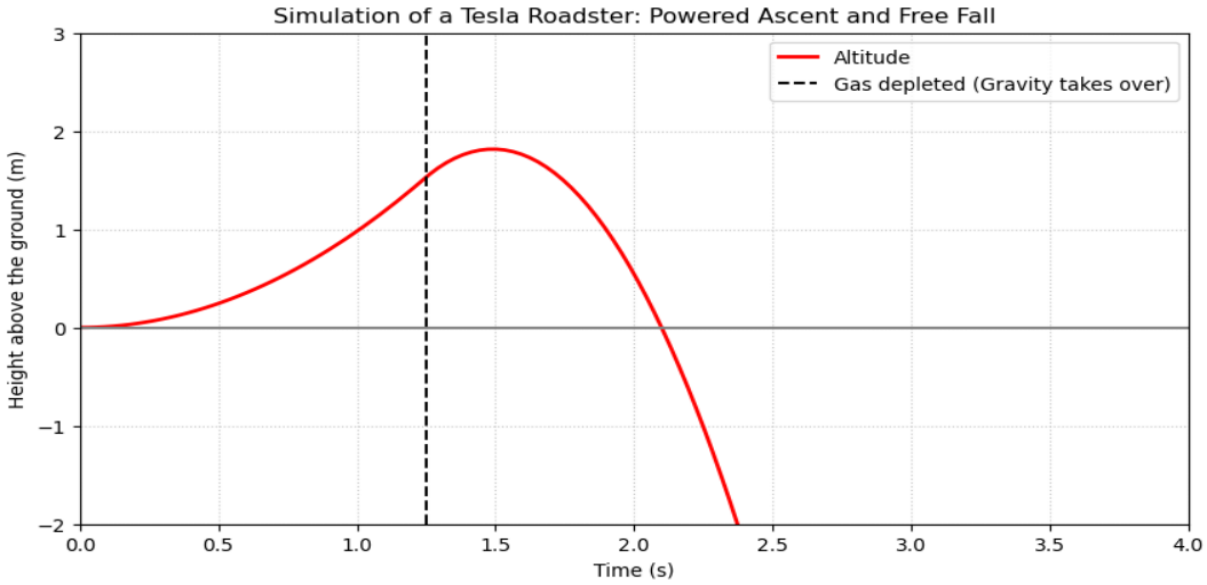


Figure 1. Height-versus-time graph

The most dramatic insight comes immediately after the gas supply depletes (marked by the dashed vertical line). With the upward thrust gone, gravity takes complete control. The

altitude curve reaches its apex and plunges downwards in a steep, accelerating fall. The simulation intentionally continues below zero altitude to show the mathematical trajectory of the free-fall crash, as ground impact is not modeled.

Ultimately, this graph serves as the definitive conclusion of the project: while the cold-gas system can provide a brief period of upward acceleration, the energy capacity of the gas tank is nowhere near sufficient for actual, sustained aviation. The Roadster is not flying; it is performing a highly controlled, ballistic jump.

Inverse Problem Solving and Mass Optimization (Code Explanation)

While previous sections focused on simulating the flight of a specific 2000 kg vehicle, this section addresses a different engineering question: *How light must the vehicle be to achieve a specific flight time?* To answer this, the code reformulates the problem into an inverse calculation. Instead of calculating time based on mass, it defines a function to calculate the required mass for a given target hover time (in this case, 10 seconds).

To find the exact mass, the script utilizes the brentq function from the scipy.optimize module. This is a powerful root-finding algorithm that searches for the specific mass value that makes the difference between the theoretical hover time and the target hover time equal to zero. By setting a search bracket between 1 kg and 5000 kg, the algorithm mathematically converges on the precise mass required, demonstrating how numerical optimization can be used to solve complex physical constraints that are difficult to isolate algebraically.

```
def mass_needed_for_hover(t_target):
    def equation(m_car_test):
        return (m_gas * v_exhaust) / (m_car_test * g) - t_target

    ideal_mass = brentq(equation, 1, 5000)
    return ideal_mass

target_time = 10.0
mass_for_10s = mass_needed_for_hover(target_time)

print(f"So that a Tesla could hover in mid-air {target_time} seconds,")
print(f"its mass must be: {mass_for_10s:.2f} kg.")

So that a Tesla could hover in mid-air 10.0 seconds,
its mass must be: 250.09 kg.
```

Code Part 5. And how much should a car weigh?

The result of the optimization reveals a stark reality. To achieve a 10-second hover using the given 100-liter gas tank, the Tesla Roadster's mass would have to be drastically reduced to **250.09 kg**. This is the approximate weight of a heavy motorcycle or a small racing quadcopter, completely stripping away the mass of a two-door sports car, its battery pack, and safety structures.

This result highlights the current physical impossibility of mass-market flying cars. Cold-gas thrusters, while reliable and used in spacecraft, have a very low energy density compared to liquid fuels or batteries. To keep a 2000 kg supercar airborne for a meaningful amount of time,

the compressed gas tank would need to be impossibly large, which would add even more weight and create a cyclical, unsolvable engineering paradox with current materials.

However, despite this mathematical impossibility for practical daily flight, this demonstration represents a monumental leap forward in automotive engineering. Just a few decades ago, active aerodynamic systems, multi-mode vacuum downforce, and integrated cold-gas thrusters were the exclusive domain of theoretical physics and aerospace research. The fact that an automotive manufacturer is now integrating these extreme, multi-disciplinary systems into a functional prototype pushes the boundaries of what is considered possible, proving that while we are not yet in the era of flying cars, the technological gap is rapidly closing.

The Engineering Paradox of Volume

After establishing the theoretical limits of the Tesla Roadster, the next logical step in our computational model was to test the scalability and practical feasibility of this technology. We wrote this specific block of code to answer a broader engineering question: *Does this cold-gas flight system work only for a highly specialized stunt vehicle, or could it be scaled for everyday passenger cars?* To find out, we applied the system's parameters to a standard production vehicle—the Alfa Romeo Stelvio—which coincidentally shares the same 2,000 kg mass as our initial Tesla model.

The code is designed as an "inverse calculation." Instead of calculating how long a given tank will last, we fixed the desired outcome: a 10-second hover. Knowing the required gas mass flow rate from our previous calculations, we determined the exact mass of gas needed for this duration. We then utilized the Ideal Gas Law backwards to calculate the physical volume required for a tank pressurized at 300 bar to hold that exact amount of gas.

Ultimately, this code block serves as a rigorous "reality check." By converting mathematical abstractions into tangible volume (liters), we can visually and numerically demonstrate the physical absurdity of the system for a daily driver. It proves that the required gas storage space physically exceeds the vehicle's dimensions, creating an unsolvable engineering paradox where the weight of the necessary tank would negate the lift it provides.

```
mass_alfa = 2000.0
hover_target = 10.0

gas_mass_needed = m_dot_required * hover_target

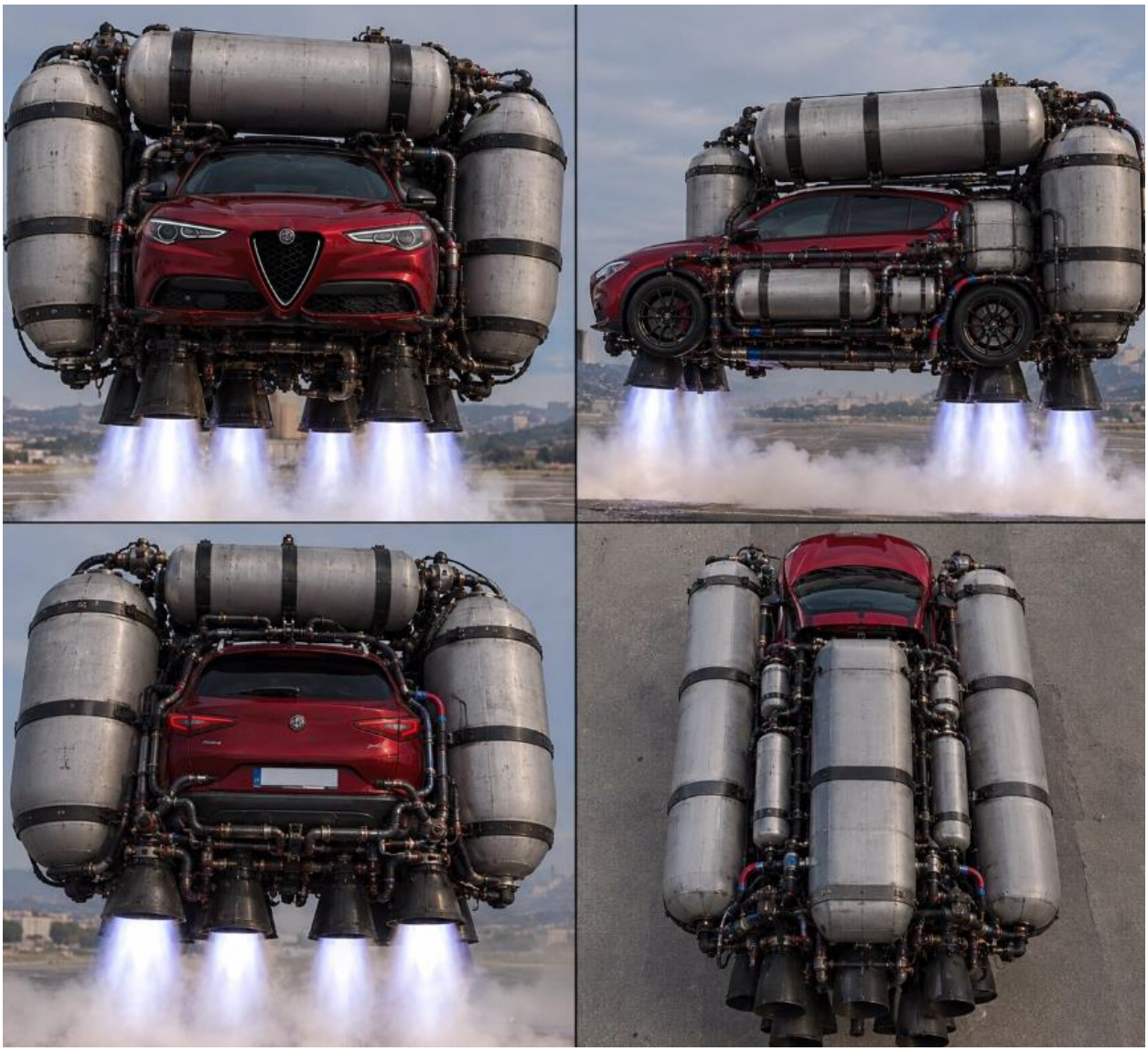
V_required = (gas_mass_needed * R_spec * T_tank) / P_tank
V_required_liters = V_required * 1000

print(f"For an Alfa Romeo Stelvio (weighing {mass_alfa} kg) to hover in mid-air for {hover_target} seconds:")
print(f"1. We would need {gas_mass_needed:.1f} kg of compressed air.")
print(f"2. The required tank volume (at 300 bar) would be {V_required_liters:.1f} liters.")
```

```
For an Alfa Romeo Stelvio (weighing 2000.0 kg) to hover in mid-air for 10.0 seconds:
1. We would need 285.2 kg of compressed air.
2. The required tank volume (at 300 bar) would be 799.7 liters.
```

Code Part 6. Flying Stelvio

When we apply our optimization model to a standard production vehicle like the Alfa Romeo Stelvio (mass ≈ 2000 kg), the mathematical reality of the cold-gas system becomes clear. To keep this vehicle hovering in mid-air for just 10 seconds, the system would require approximately 285 kg of compressed gas, which translates to a storage tank volume of 799.7 liters at 300 bar pressure.



Picture 1. An AI-generated illustration of the equipment for a flying Stelvia

To put this into perspective, the standard trunk capacity of an Alfa Romeo Stelvio is only about 525 liters. This means that the gas tank alone would completely consume the entire cargo space of the SUV, extending into the passenger cabin. Furthermore, this calculation only accounts for the volume of the gas; it does not account for the immense structural weight of a carbon-fiber pressure vessel capable of holding 300 bar.

Adding the weight of such a massive tank would significantly increase the vehicle's total mass, which in turn would require even more thrust and even more gas to achieve the same 10-second hover. This creates a closed-loop engineering paradox (a "Catch-22"). Consequently, we can mathematically prove that while cold-gas thrusters are perfect for brief, high-intensity stunts on ultra-lightweight prototypes, they are fundamentally incompatible with heavy, practical passenger vehicles.

From Ballistic Jump to Sustained Flight

While the Tesla Roadster demonstrated a fascinating, albeit brief, brute-force ballistic jump, the concept of a true flying car has long been a staple of science fiction. Without a doubt, the most famous flying automobile in cinematic history is the DeLorean DMC-12 from *Back to*

the Future. If the Tesla serves as a test of current thermodynamic and material limits, the DeLorean serves as a study of theoretical engineering and control dynamics.

In the film's lore, Doc Brown does not rely on simple compressed gas or aerodynamic wings. Instead, he installs two revolutionary technologies: a **Hover Conversion system** and **Mr. Fusion**. The hover conversion physically rotates the vehicle's wheels into horizontal thruster pads, allowing the car to push directly against the ground for continuous lift. Mr. Fusion, a compact home-energy reactor, provides an immense and continuous power supply. This shifts the paradigm from a disposable gas tank to a sustained, highly maneuverable flight system.



Picture 2. Back to the Future flying time machine

```
m_delorean = 1500.0
g = const.g
F_weight = m_delorean * g
v_exhaust_del = 800.0 # м/с

P_hover_required = 0.5 * F_weight * v_exhaust_del
print(f"Ideal power required for the DeLorean to hover: {P_hover_required / 1e6:.2f} Megawatts (MW)")

def delorean_flight(t, state):
    y, v = state
    if t < 2.0:
        thrust = F_weight * 1.2
    elif t < 8.0:
        thrust = F_weight * 1.0
    else:
        thrust = F_weight * 0.8
    drag = 0.5 * 1.225 * 1.0 * v * np.abs(v)
    a = (thrust - m_delorean * g - drag) / m_delorean
    return [v, a]
```

Code Part 7. Delorean Flight

To mathematically model the DeLorean's flight, we must return to Newton's Second Law and the scipy.integrate.solve_ivp solver. However, the ordinary differential equation (ODE) we write for this simulation is fundamentally different from the one we used for the Tesla.

For the Tesla, the thrust was a binary state: it was either fully "on" until the gas ran out, and then instantly "off." The DeLorean, however, is a controllable aircraft. Therefore, we write a **piecewise thrust function** within our ODE. Instead of a constant value, the thrust variable is dynamically calculated based on the phase of the flight. The equation evaluates the net acceleration by subtracting gravity and aerodynamic drag from a constantly adjusting thrust value. We define the thrust to be 120% of the vehicle's weight during takeoff, exactly 100% to maintain a stable hover, and 80% to execute a controlled descent.

By structuring our code this way, we are simulating a real-world flight control system (like that of a modern drone or VTOL aircraft). The mathematical integration allows us to see how the car responds to changing forces. It mathematically proves that maintaining a 1.5-tonne vehicle in mid-air indefinitely requires not just an enormous power source (Mr. Fusion), but a continuous, dynamic balance of forces. The DeLorean's flight is not a stunt; it is a simulated act of engineering control.

Assuming an idealized high-speed thruster (800 m/s), the DeLorean requires nearly 6 Megawatts of power. This mathematically justifies the need for Mr. Fusion; no conventional battery could sustain this discharge.

```
t_span = (0, 12)
t_eval = np.linspace(t_span[0], t_span[1], 500)
initial_state = [0.0, 0.0]

sol_del = solve_ivp(delorean_flight, t_span, initial_state, t_eval=t_eval)
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(sol_del.t, sol_del.y[0], color='gray', linewidth=3, label='DeLorean DMC-12 (Hover Mode)')

ax.axvspan(0, 2, color='lightgreen', alpha=0.3, label='Takeoff Phase')
ax.axvspan(2, 8, color='lightblue', alpha=0.3, label='Hovering Phase')
ax.axvspan(8, 12, color='lightcoral', alpha=0.3, label='Landing Phase')
ax.set_title('DeLorean Hover Mode Simulation (Back to the Future Physics)', fontsize=14, fontweight='bold')
ax.set_xlabel('Time (s)', fontsize=12)
ax.set_ylabel('Altitude (m)', fontsize=12)
ax.grid(True, linestyle=':', alpha=0.7)
ax.legend(loc='upper left')
plt.show()
```

Code Part 8. Graph building

This block of code executes the core numerical simulation and visualizes the flight phases.

- t_span and t_eval define the time domain of the simulation, running from 0 to 12 seconds and evaluating 500 data points to ensure a smooth curve.
- initial_state = [0.0, 0.0] establishes the boundary conditions: the DeLorean starts at ground level (altitude = 0) with zero initial vertical velocity.
- The solve_ivp function integrates the Ordinary Differential Equation (ODE) defined earlier using a Runge-Kutta method. It returns sol_del, an object containing the time array (sol_del.t) and the calculated state variables (altitude at sol_del.y[0] and velocity at sol_del.y[1]).
- The plotting section utilizes ax.axvspan to visually segment the flight into three distinct phases (Takeoff, Hovering, Landing) using semi-transparent color bands. This is an excellent technique for scientific visualization, as it allows the viewer to instantly correlate the vehicle's kinematic behavior with the specific thrust logic applied by the flight computer.

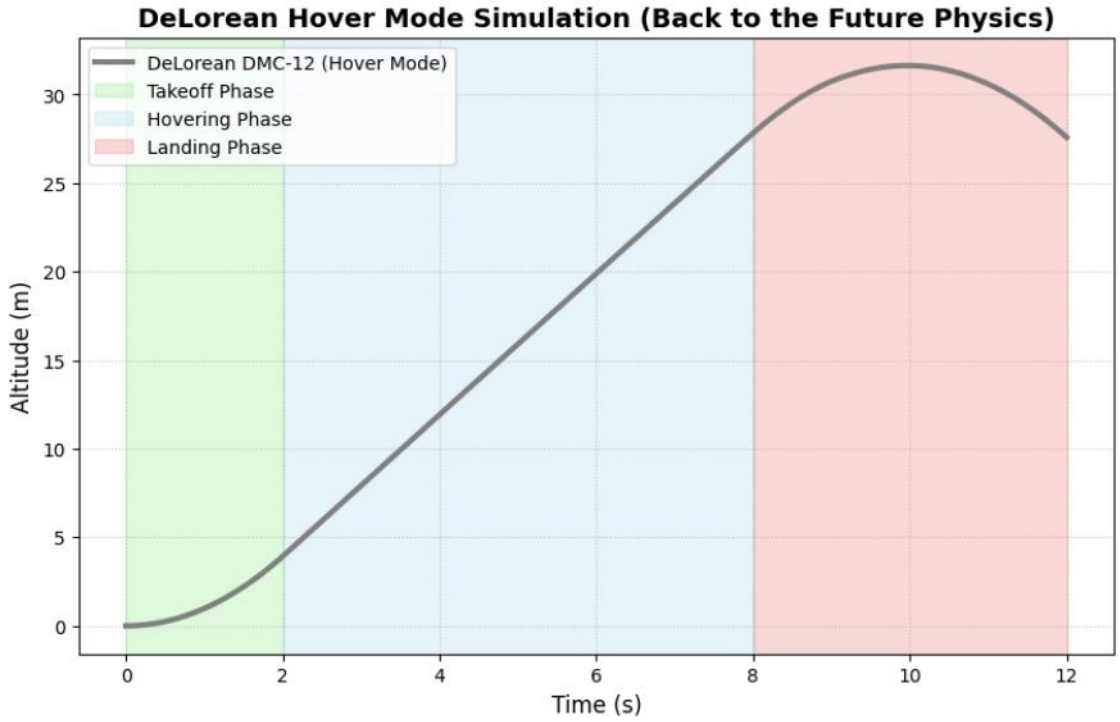


Figure 2. Ideal power required for the DeLorean to hover: 5.88 Megawatts (MW)

Kinematic Analysis of the Hover Mode (Graph Interpretation)

The resulting altitude-time graph perfectly illustrates the transition between different flight regimes, but it reveals a fascinating physical phenomenon regarding inertia.

- **Phase 1 (Takeoff, 0-2s, Green):** The thrust is 120% of the vehicle's weight. The net force is positive, resulting in constant positive acceleration. The altitude curve is a steep parabola, showing rapid upward movement.
- **Phase 2 (Hovering, 2-8s, Blue):** *Note the linear trajectory here.* Because the thrust is exactly 100% of the weight, the net force is zero. Therefore, acceleration is zero. However, Newton's First Law dictates that the car maintains the upward velocity it gained during takeoff. It is not "hovering" in place; it is climbing at a constant speed (cruising).
- **Phase 3 (Landing, 8-12s, Red):** Thrust drops to 80% of the weight, creating a net downward force. The upward velocity begins to decelerate, the curve reaches its peak (the apex of the flight), and the vehicle begins a controlled descent back to the ground.

The resulting graph reveals a crucial physical phenomenon regarding inertia and control. During the designated "Hovering Phase" (the blue section of the graph), the line is not flat; it continues to rise linearly. This mathematically demonstrates Newton's First Law: because the thrust exactly equals the vehicle's weight, the net force is zero, meaning acceleration is zero. However, the car maintains the upward velocity it gained during takeoff. In real-world engineering (such as drone or VTOL flight control), a true stationary hover requires the flight computer to constantly modulate the thrust slightly below 100% to counteract inertia. This proves that the DeLorean's flight requires a highly advanced, continuous control system—a stark contrast to the Tesla's brute-force ballistic approach.

Our simulation of the Tesla Roadster mathematically proved that the "SpaceX package" is not a step toward sustainable aviation, but rather a brilliant, albeit brief, ballistic maneuver. The thermodynamic constraints of cold-gas thrusters mean that the mass-to-thrust ratio quickly depletes the gas supply, reducing the vehicle to a projectile. Furthermore, our inverse calculation

proved an engineering paradox: scaling this system for a standard SUV (like the Alfa Romeo Stelvio) would require a gas tank larger than the vehicle's cabin, creating an unsolvable weight-to-lift cycle.

Conversely, the theoretical simulation of the DeLorean DMC-12's Hover Mode highlighted the vast difference between a gas-powered jump and true, sustained flight. To maintain a controllable hover, an aircraft requires continuous, dynamic adjustments of thrust to overcome gravity and drag. This requires a continuous power source (like Mr. Fusion) capable of delivering megawatts of energy, proving that Doc Brown's fictional engineering is theoretically sound, but practically out of reach for current battery technology.

Our simulation reveals a fascinating truth: the fictional 'Hover Mode' is actually a 'Cruise Mode'. To truly hover in place, Doc Brown's flight computer would have to reduce thrust to exactly 99% of the weight to cancel out the upward inertia.

Ultimately, this project demonstrates the true power of **Data Modelling**. Through the use of NumPy for vectorized thermodynamic calculations, SciPy for solving Ordinary Differential Equations and optimizing parameters, and Matplotlib for advanced visualization (from 3D parameter spaces to phase portraits), we transformed a cinematic illusion into a strict scientific reality. We proved that while physics imposes hard limits on what cars can do today, computational modeling is the most powerful tool engineers have to explore the boundaries of what might be possible tomorrow.

Course Materials & Data Modelling

1. **Vasi, S. (2025).** *Data Modelling [A001106]: Introduction to Modeling and Data Analysis*. University of Messina (UniMe), Department of Mathematical and Computer Sciences, Physical Sciences and Earth Sciences. *(Lecture Notes 01)*
2. **Vasi, S. (2025).** *Data Modelling: Numerical and Physical Analysis with NumPy*. University of Messina (UniMe). *(Lecture Notes 03)*
3. **Vasi, S. (2025).** *Data Modelling: Optimization, Integration, and Differential Equations with SciPy*. University of Messina (UniMe). *(Lecture Notes 06)*
4. **Unpingco, J. (2021).** *Python Programming for Data Analysis*. Springer Cham. *(Recommended textbook for the course, used as a reference for DataFrames, NumPy arrays, and SciPy functions).*
5. **Kinser, J. M. (2022).** *Modeling and Simulation in Python*. Chapman and Hall/CRC. *(Used for the theoretical background on numerical simulations and ODE solving).*

Classical Physics & Mechanics

1. **Landau, L. D., & Lifshitz, E. M. (1976).** *Mechanics (Course of Theoretical Physics, Vol. 1)*. Pergamon Press (Original Russian Edition: Наука).
(Used for the theoretical foundation of Newton's laws, kinematics, and equations of motion in non-inertial frames).
2. **Irodov, I. E. (1981).** *Problems in General Physics*. Mir Publishers (Original Russian Edition: Высшая школа).
(Used as a reference for solving complex kinematic problems and understanding the dynamics of bodies moving under the influence of gravity and drag).
3. **Sivukhin, D. V. (1980).** *General Physics Course, Vol. 2: Thermodynamics and Molecular Physics*. Nauka (Original Russian Edition).
(Used for the thermodynamic analysis, specifically the adiabatic expansion of the gas and the Saint-Venant–Wantzel equation).